



LISTA DE EXERCÍCIOS 7

GABARITO...

OBS. Resoluções em sala de aula podem englobar mais pontos de vista e atributos do que apenas as respostas simplificadas apresentadas no gabarito.

1. Liste três exemplos de *deadlocks* que não estão relacionados a um ambiente computacional.

Resposta: 1) Dois carros cruzando uma ponte com mão única em lados opostos; 2) uma pessoa descendo uma escada enquanto outra pessoa está subindo uma escada; 3) Dois trens viajando em sentidos opostos em uma mesma estrada de ferro.

2. Suponha que um sistema está em um estado inseguro. Mostre que é possível para o processo completar as suas execuções sem entrar em um estado de *deadlock*.

Resposta: Um estado inseguro pode não necessariamente levar a um *deadlock*. Isso só significa que não se pode garantir que um *deadlock* não irá acontecer. Assim, é possível que um sistema em um estado inseguro possa permitir que todos os processos sejam completados sem que ocorra um *deadlock*. Como exemplo, considere a situação onde um sistema possua 12 recursos alocados entre os processos P_0 , P_1 , e P_2 . Os recursos são alocados de acordo com a seguinte política:

	Max	Current	Need
P_0	10	5	5
P_1	4	2	2
P_2	9	3	6

No momento, existem dois recursos disponíveis. Este sistema está em um estado inseguro considerando que P_1 poderia completar, assim, liberando um total de quatro recursos. Mas, não se pode garantir que os processos P_0 e P_2 possam completar. Entretanto, é possível que um processo possa liberar recursos antes de requisitar mais recursos. Por exemplo, o processo P_2 poderia liberar um recurso, aumentando assim o número total de recursos para cinco. Isso permite que o processo P_0 complete, o que liberaria um total de nove recursos, de modo a permitir que o processo P_2 também complete.

3. Considere a seguinte representação de um sistema:

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	<i>A B C D</i>	<i>A B C D</i>	<i>A B C D</i>
P_0	0 0 1 2	0 0 1 2	1 5 2 0
P_1	1 0 0 0	1 7 5 0	
P_2	1 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 6	

Responda as seguintes questões usando o algoritmo do banqueiro:

a. Qual é o conteúdo da matriz *Need*?

Resposta: Os valores de *Need* para o processo P_0 até o P_4 respectivamente são (0, 0, 0, 0), (0, 7, 5, 0), (1, 0, 0, 2), (0, 0, 2, 0) e (0, 6, 4, 2).

b. O sistema está em um estado seguro?

Resposta: Sim. Com a matriz *Available* = (1, 5, 2, 0), o processo P_0 ou o processo P_3 poderiam executar. Uma vez que o processo P_3 execute, ele libera seus recursos, os quais permitem que todos os outros processos existentes executem.

c. Se uma requisição do processo P_1 chegar para (0, 4, 2, 0), a requisição pode ser garantida imediatamente?

Resposta: Essa questão pode possuir duas interpretações. Se a requisição (0, 4, 2, 0) é simplesmente uma solicitação parcial de recursos de P_1 , então é possível alocar imediatamente esses recursos. Afinal, a matriz *Available* possui esses recursos disponíveis para alocação imediata. Por outro lado, se essa requisição representar recursos adicionais sendo solicitados pelo processo P_1 , a resposta seria não. Com a matriz *Max* para P_1 assumindo o valor (1, 11, 7, 0) (i.e. recursos considerados como adicionais aos que P_1 já havia solicitado), a ordem de execução para se tentar atender uma requisição “imediata” de P_1 seria $P_3 \rightarrow (P_2 \text{ ou } P_4) \rightarrow P_1$, ou seja, não é possível executar P_1 imediatamente.

- Um método possível para prevenir *deadlocks* é ter um único recurso de alta-ordem que deve ser requisitado antes de qualquer outro recurso. Por exemplo, se múltiplas *threads* tentarem acessar os objetos de sincronização A...E, *deadlock* é possível. (Tais objetos de sincronização podem incluir *mutexes*, semáforos, variáveis de condição e afins.) É possível prevenir o *deadlock* pela inclusão de um sexto objeto F. Sempre que uma *thread* desejar adquirir o *lock* de sincronização para qualquer objeto A...E, ela deve primeiro adquirir o *lock* para o objeto F. Esta solução é conhecida como “contenção” (*containment*): os *locks* para os objetos A...E estão contidos dentro do *lock* para o objeto F. Por que esse esquema defavorece a ocorrência de *deadlocks*?

Resposta: Esta, provavelmente, não é uma boa solução pois resulta em um escopo muito grande (i.e. permite o travamento de objetos A...E indiscriminadamente). É melhor definir uma política de travamento com um escopo tão pequeno quanto possível (favorece paralelismo).

- Considere um sistema computacional que execute 5.000 tarefas por mês sem prevenção ou coibição de *deadlock*. *Deadlocks* ocorrem cerca de duas vezes por mês, e o operador deve terminar e re-executar cerca de 10 tarefas por *deadlock*. Cada tarefa vale cerca de R\$ 2 (em tempo de CPU), e as tarefas terminadas em geral, estão aproximadamente na metade de sua execução quando são abortadas. Um programador de sistema estimou que um algoritmo de coibição de *deadlock* (como o algoritmo do banqueiro) poderia ser instalado no sistema com um aumento no tempo médio de execução por tarefa de cerca de 10%. Como a máquina possui atualmente 30% de tempo ocioso, todas as 5.000 tarefas executadas por mês ainda poderiam ser executadas, embora o *turnaround time* aumentaria cerca de 20% em média.

a. Quais são os argumentos favoráveis a instalação do algoritmo de coibição de *deadlock*?

Resposta: Um argumento para instalar um algoritmo de coibição de *deadlock* no sistema é que poderia-se assegurar que *deadlock* nunca irá ocorrer. Além disso, apesar do aumento no *turnaround time*, todos os 5.000 processos (*jobs*) poderiam executar. Se considerarmos o impacto financeiro de não execução de um subconjunto dessas tarefas (ex. 20 tarefas) e uma análise de custo, talvez o aumento no valor de processamento de cada tarefa fosse justificável (vide próxima questão). Ganho possível: DEMISSÃO (ou realocação para outra atividade) DO FUNCIONÁRIO RESPONSÁVEL POR REINICIAR AS TAREFAS. Além disso, deve-se avaliar o impacto para o sistema de não execução de cada uma das 20 tarefas (i.e. alguma delas é crucial para o sistema?).

b. Quais são os argumentos contrários a instalação do algoritmo de coibição de *deadlock*?

Resposta: Um argumento contrário para a instalação do algoritmo de coibição de *deadlock* é resultante da observação que *deadlocks* ocorrem com pouca frequência, e eles custam muito pouco quando ocorrem (aprox. R\$ 40,00 para as 20 tarefas em *deadlock*, segundo o cenário descrito). Com um custo de cerca de 10% de aumento de tempo de execução em relação ao *turnaround time* por tarefa e o custo adicional previsto de 20% sobre o *turnaround time*, esse sistema com a característica de coibição de *deadlock* teria um custo adicional mensal de R\$ 2.000,00. Em termos de análise de custo, parece ser um aumento desvantajoso.

6. Um sistema poderia detectar *starvation* em alguns de seus processos? Caso sua resposta seja “sim”, explique como. Se você respondeu “não”, explique como o sistema pode lidar com o problema de *starvation*.

Resposta: *Starvation* é um tópico difícil de ser definido, considerando que pode ter diferentes significados em sistemas diferentes. Para o propósito dessa questão, definiremos *starvation* como sendo a situação na qual um processo pode esperar além de um intervalo de tempo razoável – talvez indefinidamente – antes de receber um recurso requisitado. “Sim”, implica na ocorrência de *starvation*. Nesse caso, irá se detectar a condição de *starvation* por meio da avaliação de um período de tempo – T – que é considerado razoável para o atendimento de uma requisição. Quando um processo requisita um recurso, um temporizador é iniciado. Se o tempo determinado T é excedido, então o processo é considerado “morto por inanição” (*starved*). Ao se detectar essa condição, o processo poderia ser colocado para execução imediatamente (ou eventualmente, em um momento “limiar” anterior a caracterização de *starvation*). Na condição “não”, pressupõe-se que não ocorre *starvation*. Se não ocorre *starvation*, isso quer dizer que a política de escalonamento possui algum mecanismo para tratar a possibilidade de ocorrência de *starvation* e essa condição nunca ocorre (e.g. o algoritmo trabalha com alguma política de *aging* e os processos à medida que vão envelhecendo vão tendo sua prioridade aumentada).

7. Considere a seguinte política de alocação de recurso. Requisições e liberações de recursos são permitidas a qualquer momento. Se uma requisição por recursos não pode ser satisfeita porque os recursos não estão disponíveis, então verificamos quaisquer processos que estejam bloqueados aguardando por recursos. Se um processo bloqueado tem os recursos desejados, então estes recursos são tomados dele e são dados para o processo requisitante. O vetor de recursos para o qual o processo bloqueado está aguardando é aumentado para incluir os recursos que foram tomados. Por exemplo, considere um sistema com três tipos de recurso e o vetor *Available* inicializado para (4,2,2). Se o processo P_0 pede por (2,2,1), ele os consegue. Se P_1 pede por (1,0,1), ele os consegue. Então, se P_0 pedir por (0,0,1), ele é bloqueado (recurso não disponível). Se P_2 pede agora por (2,0,0), ele consegue o recurso disponível (1,0,0) e aquele que foi alocado a P_0 (já que P_0 está bloqueado). O vetor de alocação de P_0 chega em (1,2,1) e seu vetor *Need* chega em (1,0,1).

a. Pode ocorrer *deadlock*? Se sua resposta for “sim”, dê um exemplo. Se sua resposta for

“não”, especifique quais condições, necessariamente não devem acontecer.

Resposta: *Deadlock* não pode ocorrer pois existe a possibilidade de preempção.

b. Pode ocorrer bloqueio indefinido? Explique sua resposta.

Resposta: Sim. Um processo pode nunca adquirir todos os recursos necessários se esses recursos forem continuamente “tomados” por uma série de requisições, assim como demonstrado pela tomada de recurso de P_0 por P_2 , principalmente se um processo pode adquirir recursos parcialmente e não iniciar sua execução de imediato.

8. É possível ter *deadlock* envolvendo um único processo com uma única *thread*? Explique sua resposta.

Resposta: Não. Isso é deduzido diretamente da análise da condição de guarda e espera.